ATRO Design & Construction

ATRO is a Kenyan design-and-construction company platform. It brings the public company website, prospective-client enquiries, project operations, and a private client project portal together around one project record.

The platform is intended to make a build easier to understand and coordinate: project progress and site evidence, decisions, approvals, documents, payment certificates, messages, and handover obligations are recorded in the same project history rather than scattered across separate dashboards.

What the platform does

Public website and enquiries

The public site explains ATRO's services, zones, process, and company, and provides a guided project enquiry. The enquiry captures project type, location, scale, contact details, and—when relevant—diaspora context. It can show an indicative cost range; that range is not a formal quotation.

Internal project operations

The `/admin` console is shared by studio staff but changes its available views and actions according to the signed-in staff member's role. It supports enquiry handling, project records, project updates and evidence, approvals, decisions, documents, client communication, calendars, and operational follow-up. The admin portfolio summarizes business health and prioritizes project exceptions; its project register can be searched, filtered by lifecycle, sorted, and paged. Selecting a project opens its command center for the detailed record and authorized changes.

Private client portal

Clients sign in at `/login` and use `/portal` to see projects associated with their account. The portal provides project progress and evidence, milestones, client approvals, decisions, payment status and documents, messages, selections and changes, site-presence coordination, and after-handover support.

Kenyan and diaspora clients share the same underlying project records and permissions. The interface adapts coordination details to the client:

- **Kenyan clients** can request a site visit and see confirmed appointments in Nairobi time.
- **Diaspora clients** can request a remote walkthrough and see relevant calls and appointments in their own configured time zone, alongside Kenya time where useful.
- **Project representatives** can be invited to one named project through the Project circle. They can read milestone/site updates, request a site visit, and submit recommendations on open decisions. The owner records every final decision; representatives cannot approve milestones, access financial records, change contracts, or invite others. Invitations are single-use, require the invited verified email, expire after 7 days, and grant 90 days of access after acceptance. Owners copy and share the invitation link themselves; the app does not send invitation email.

Roles and responsibilities

Role	Main responsibility	Typical access
Admin / owner	Own the portfolio and business-level decisions.	Portfolio and business overview; project operations; lifecycle transitions; financial actions such as payment certification and retention release; staff access administration.
Office manager	Coordinate the studio's day-to-day project operations.	Project and enquiry coordination; shared calendar and call scheduling; project follow-up. Admin-only financial, lifecycle close-out, business overview, and access-administration actions remain restricted.
Staff	Work the operational queue and maintain project records.	Enquiries, projects, project updates, evidence and coordination; read the shared calendar and manage personal follow-up reminders. Call scheduling and admin-only actions are restricted.
Kenyan client	Follow their project, review evidence, make decisions, and coordinate locally.	Their own project records and client actions, including approvals, decisions, payments, documents, messages, site visits, and after-handover items.
Diaspora client	Do the same client work remotely and asynchronously.	The same client records and rights, with timezone-aware coordination, remote walkthroughs, and abroad-specific guidance.
Project representative	Help coordinate on a project for its owner.	Only explicitly shared project updates, site-visit requests, and decision recommendations. No final approvals, payments, contract actions, other projects, or access administration.

Staff roles are allowlisted. Admins manage staff access; office managers do not receive access to the staff-access administration. Server-side authorization protects reads and writes: hiding a button in the interface is not treated as permission enforcement.

The shared staff calendar has a navigable month view and a selected-day agenda using Nairobi time. Scheduled calls and confirmed visits or walkthroughs appear on their dates; calls without a time and presence requests awaiting a response remain separate queues. Office managers and admins can schedule calls; presence requests are answered from the project workflow. Personal follow-ups remain reminders rather than calendar appointments.

Clients can manage their project-update contact details separately from the verified sign-in identity. Staff can set a preferred console name and work focus; their studio role and permissions remain owner-controlled and are not profile settings.

End-to-end project flow

1. **A prospect discovers ATRO.** They learn about the services and process on the public site and submit a guided enquiry, or contact the studio directly.
2. **The studio follows up.** Staff review the enquiry inbox, claim or progress enquiries through the funnel, and record follow-up. The indicative estimate helps start a conversation; it is not a quote or a confirmed contract.
3. **A project record is prepared.** An authorized studio user creates or prepares the project record, assigns its client account, and records the scope, programme, milestones, payment schedule, documents, approvals, and decisions as the project is agreed and work proceeds. A client may also start a project record in the portal; generated planning information should be checked and confirmed with the studio before it is treated as an agreed scope or commercial commitment.
4. **The client signs in.** The client can use Google, a passwordless email code, or a phone one-time code. The verified email or phone must match the contact details ATRO has on the project so the portal can find and claim it. Project reads and writes are checked against project ownership on the server. An owner may separately invite a trusted representative to a single project. The invitation must be accepted by a signed-in account with the matching verified email; the representative's limited access expires and can be revoked by the owner.
5. **Work is delivered and recorded.** Staff maintain milestones and evidence, publish updates, manage approval records and client decisions, maintain project documents and payment records, and coordinate calls or visit/walkthrough requests. Follow-up dates and weekly checklists help the team keep work visible.
6. **The client reviews and responds.** Depending on the item, the client can review evidence, approve a milestone marked ready, record a decision, pay a certified instalment through Paystack, review project documents, message the studio, or request a site visit or walkthrough. Client choices are recorded in the project history. A project representative may request a site visit and suggest an answer to an open decision; the owner must record any final decision or approval.
7. **The project moves through its lifecycle.** The studio tracks pre-construction, active build, handover, and the defects period. Build completion is not by itself a record of handover or close-out. Closing and archiving are admin-only and subject to lifecycle checks.
8. **After handover, obligations remain visible.** Defect reports and rectification, the defects-period end, and retention are tracked until the required close-out work is completed.

Project lifecycle states

- **Pre-construction** — preparation, approvals, and registration.
- **Active build** — work is underway.
- **Handover** — completion inspection and handover records.
- **Defects period** — post-handover obligations remain active.
- **Closed** — close-out conditions have been met.
- **Archived** — retained for reference and removed from live operational queues.

Transitions are recorded with an actor, time, and reason. Handover requires all tracked milestones to be complete; closing requires the relevant defects and retention obligations to be resolved. A project being

100% complete by progress alone does not automatically close it.

Local development

Requirements

- Node.js compatible with the versions required by the dependencies in `package.json`
- npm

Start the app

```
npm install
npm run dev
```

The development server runs on port `8080`. On startup, the app uses an embedded PGlite database and applies the SQL migrations in `migrations/` automatically. The local PGlite database is in-memory: its data resets when the server process restarts.

`npm run dev`, `npm run build`, and `npm run preview` use `scripts/with-app-env.mjs`. It reads optional `VITE_`-prefixed string flags from `.grok/app-env.json`; explicit process environment variables take precedence. This file is for build flags, not secrets.

Authentication and database configuration

Phone one-time-code, passwordless email-code, and Google sign-in are provided through Supabase Auth. Configure `SUPABASE_URL` and `SUPABASE_ANON_KEY` for the server and the corresponding `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` values for the browser. Enable Phone and Email providers in Supabase Auth and configure the project to deliver phone and email verification codes. The email OTP template must include Supabase's `{{ .Token }}` value.

To enable Google sign-in, enable the Google provider under Supabase Auth and enter the OAuth client ID and secret there. In Google Cloud, register the Supabase project's Auth callback URL as an authorized redirect URI. Add the app's local and deployed `/login` URLs to Supabase's redirect URL allowlist so OAuth can return to the portal.

For local development, copy `.env.example` to `.env` and fill in the values. The local `.env` file is loaded by the `dev`, `build`, and `preview` wrapper and is ignored by Git. Never add service-role keys, database passwords, or provider secrets to `VITE_` variables; Vite exposes those values to the browser.

Client payments use Paystack-hosted checkout for certified project instalments. Set the server-only `PAYSTACK_SECRET_KEY` and `PAYSTACK_WEBHOOK_SECRET`, and set `ATRO_SITE_URL` to the canonical site origin. Configure Paystack's webhook URL as `https://<your-site>/api/paystack/webhook`. Keep both Paystack keys out of `VITE_` variables and browser code. The payment callback verifies the reference directly with Paystack; a browser redirect alone never marks an instalment paid.

Staff invitations are created from the admin dashboard's Access section. Configure `RESEND_API_KEY`, a verified `RESEND_FROM` address, and the canonical `ATRO_SITE_URL` to email the `/staff` link. WhatsApp invitations open a pre-filled `wa.me` message for an admin to review and send; they do not require WhatsApp Business API credentials. In either case, the recipient must sign in with the invited email and accept before staff access is granted.

Paystack settles to the merchant bank account configured in Paystack. ATRO's project allocations are an internal ledger, not escrow or separate bank accounts. Project expense records reserve only that project's Paystack-verified receipts; an independent admin must approve an expense before staff can record an external bank-transfer reference as paid. The app does not initiate outgoing Paystack transfers.

Project-message attachments use a private Supabase Storage bucket, created by the server on the first upload. They support JPEG, PNG, WebP, PDF, DOCX, XLSX, MP4, and WebM files, up to 5 files per message and 10 MB per file. Configure `SUPABASE_SERVICE_ROLE_KEY` on the server for signed uploads and private file access. Never expose this key with a `VITE_` prefix. Files are uploaded directly to private storage; project ownership or staff access is checked before signed upload/download links are issued. Local development without Supabase can still use text messaging, but file attachments require Supabase configuration. Users can delete their own messages; the thread keeps a "Message deleted" record marker and stored attachment bytes are removed. A small emoji picker is available in both client and staff composers.

For a persistent PostgreSQL database, set `DATABASE_URL` to the database connection string. The app uses Neon/PostgreSQL when it is set; otherwise it uses the local PGlite fallback. The production build runs pending database migrations when `DATABASE_URL` is present. Do not commit credentials or secret environment files.

`migrations/schema.sql` is the canonical consolidated schema for a new Supabase database. Run it once in the Supabase SQL Editor, or let the app apply it to a fresh database. It records its schema version so the app does not replay it. Existing databases with the previous numbered migration records skip the bootstrap and continue to accept future numbered migrations. For future schema changes, continue with `0033_*.sql` and later migration numbers; do not edit the bootstrap to perform an upgrade on an existing database.

Useful commands

```
npm run typecheck
npm run lint
npm run build
npm run db:migrate
```

`npm run build` creates the production bundle, applies configured database migrations, and copies PGlite assets. Without `DATABASE_URL`, the migration script skips the external database; the local PGlite backend applies migrations when the app starts.

Technical overview

- **Frontend and routing:** React, TypeScript, Vite, and TanStack Router/Start.
- **UI and server state:** Tailwind CSS, TanStack Query, and reusable components under `src/components/`.
- **Data:** PostgreSQL-compatible SQL through a shared adapter in `src/lib/db.ts`, backed by Neon when configured or PGlite for local development.
- **Authentication:** Supabase phone OTP, with server-side session verification and role checks.
- **Schema:** `migrations/schema.sql` is the consolidated baseline; future incremental changes are numbered SQL migrations. Applied versions are tracked in `_migrations`.
- **Main application areas:** public pages in `src/routes/`, the client portal at `/portal`, staff and owner operations at `/admin`, and shared business rules/server functions in `src/lib/`.

Current scope and limitations

- The role and dashboard implementation is an operational first pass; validate workflows and terminology with representative staff and clients before relying on them as a substitute for policy or training.
- Follow-up reminders and weekly checklist items are in-app records. They do not imply an email, SMS, push-notification, or scheduled-job reminder service.
- The document and approval completeness summary reflects records entered for a project. It is not an automatically generated legal, statutory, or project-specific requirements matrix.
- Project data, schedules, estimates, and payment details must be reviewed by the responsible ATRO staff. The platform records project work; it does not itself certify construction quality, approve statutory submissions, or execute bank transfers.

Project documentation

See IMPLEMENTATION_PLAN.md for the role-based roadmap, design decisions, and validation criteria.